

Índice

1. Introducción.....	1
2. Instalación y configuración de PostGis	1
3. Bases de datos espaciales en PostgreSQL	8
3.1. Datos espaciales.....	9
4. Ejemplo 1: Geometría que contiene puntos y líneas.....	12
4.1. Creación e inserción de objetos.....	12
4.2. Consultas.....	13
4.2.1. Consulta del vecino más próximo	13
4.2.2. Función ver().....	14
4.3. El módulo principal para el ejemplo 1	14
5. Ejemplo 2: Geometría que contiene puntos, líneas y polígonos.....	15
6. Ejercicios propuestos	16

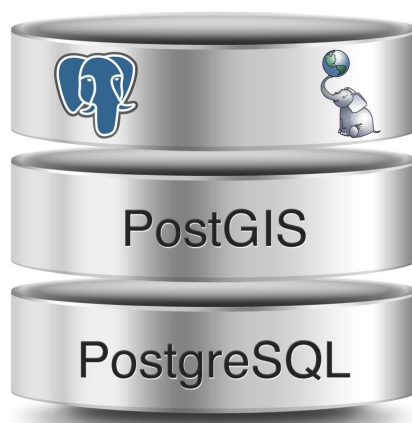
1. Introducción

En esta sesión se continúa usando el entorno de trabajo de las sesiones de prácticas anteriores. Se utilizará el gestor de base de datos ya instalado (**Postgresql**) y el entorno que facilita toda la gestión de consultas (**pgAdmin IV**).

- PostgreSQL – (*Ya instalado en prácticas anteriores*)
- pgAdmin IV – (*Ya instalado prácticas anteriores*)
- PostGis – (A instalar en esta sesión)

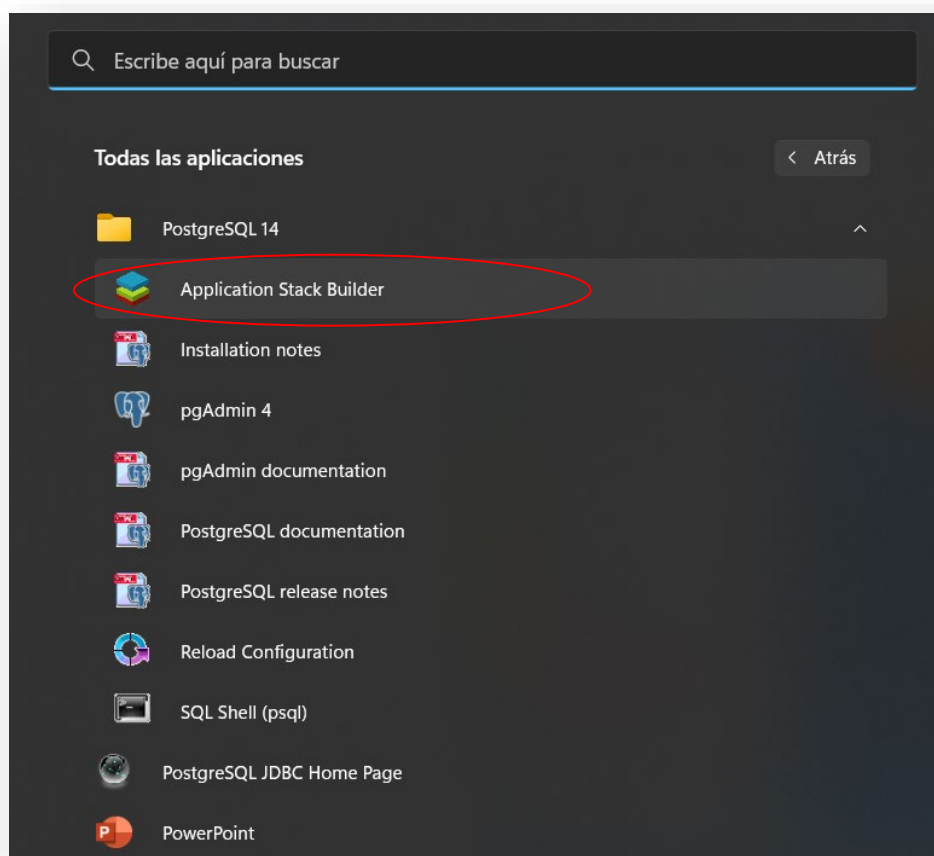
2. Instalación y configuración de PostGis

Una base de datos espacial es una colección de datos acerca de objetos localizados en un área determinada, organizados en de tal forma que puede servir eficientemente a una o varias aplicaciones. **PostGis** permite la gestión de información espacial, proporcionando funciones que facilitan el almacenamiento, la recuperación, la actualización y las consultas. Se basa en el **modelo objeto-relacional** para la representación geométrica de una característica espacial. En realidad, **PostGIS** es un módulo que añade soporte de objetos geográficos a PostgreSQL, convirtiéndola en una base de datos espacial para su utilización en Sistema de Información Geográfica. Se publica bajo la Licencia Pública General de GNU.

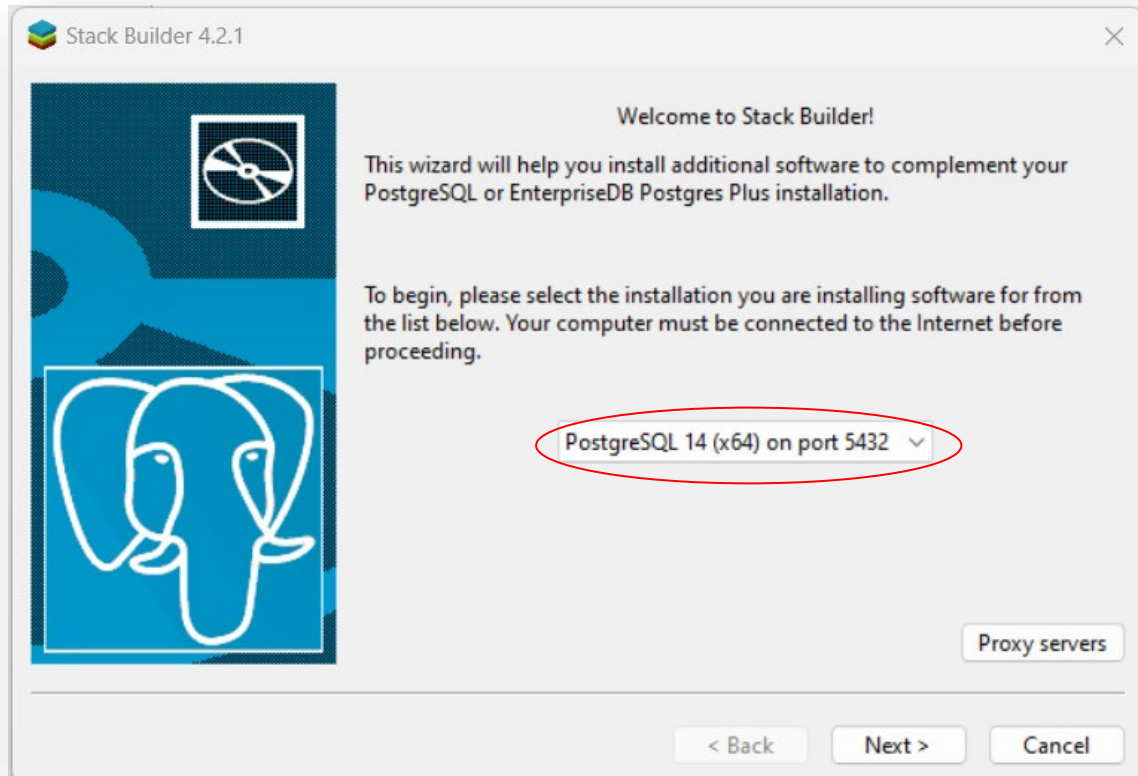


El primer paso consiste en instalar PostGis, en la versión estable que se encuentre disponible (en este caso, a la hora de redactar este guion, es la versión 2.1.3). Para ello, se deben seguir los siguientes pasos:

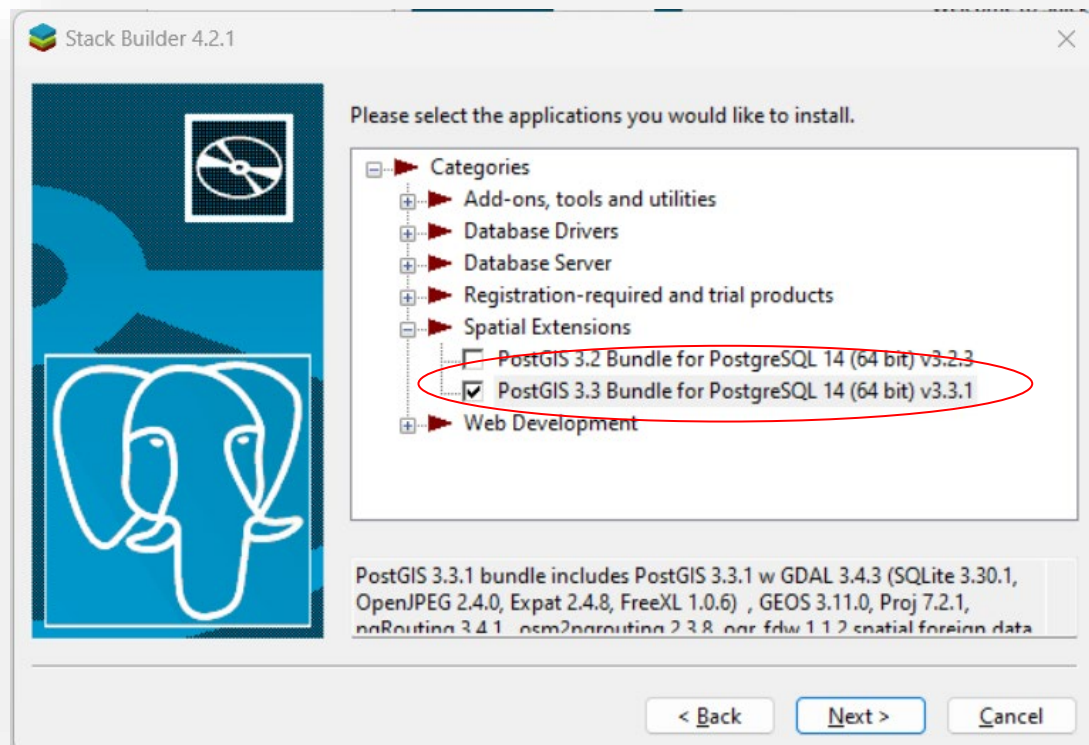
Inicio | Todas las aplicaciones | PostgreSQL 14 | Application Stack Builder



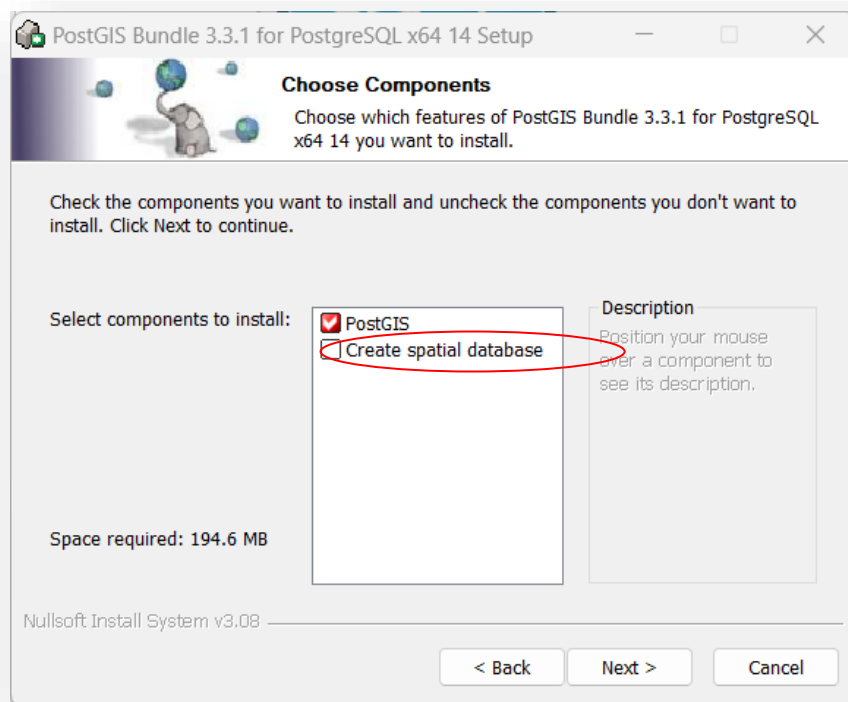
En la siguiente pantalla se selecciona la instalación de **PostgreSQL** sobre la que se va a trabajar, en nuestro caso **PostgreSQL 14** en el puerto 5432, y se pulsa **Next**.



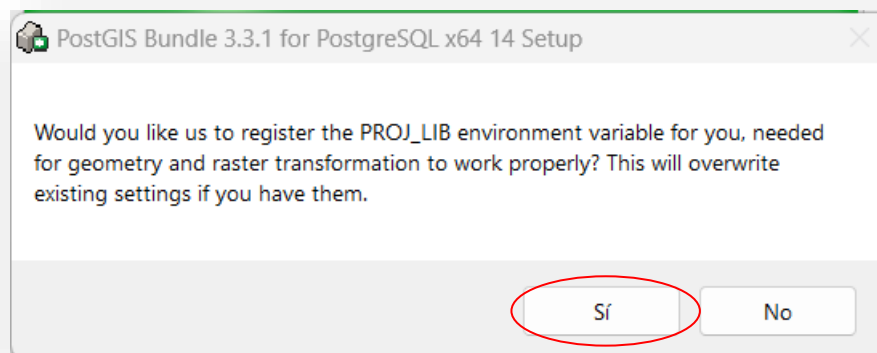
A continuación, se debe seleccionar la extensión que se desea instalar. Para ello, se despliega **Spatial Extensions** y se selecciona **PostGIS 3.3 Bundle for PostgreSQL 14 (64 bit) v3.3.1** (se selecciona una versión estable de la extensión espacial, preferiblemente la última, aunque para los ejercicios del guion no es necesario una versión avanzada):



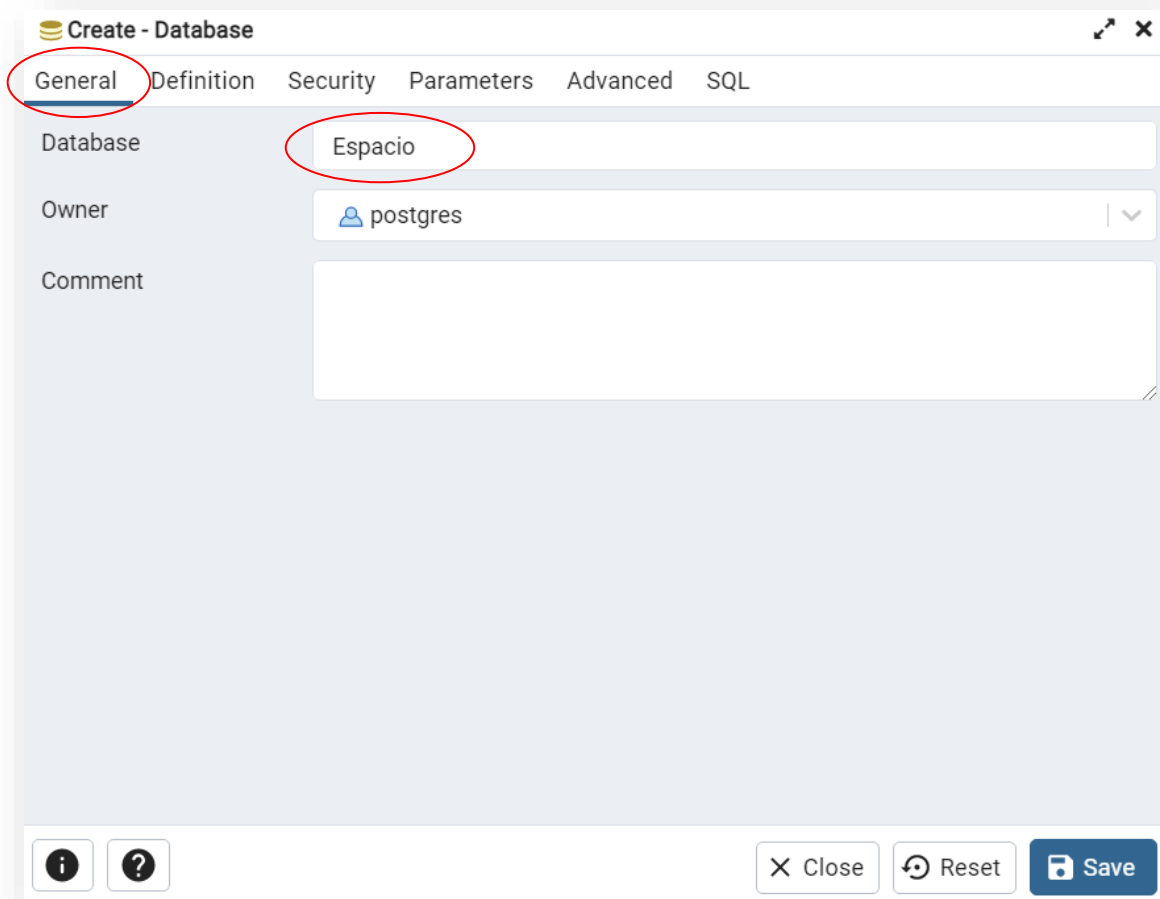
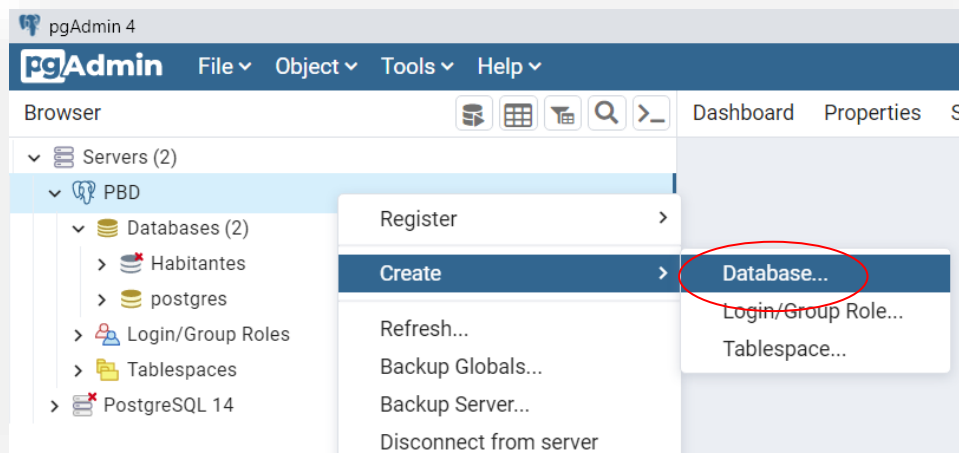
Pulsando **Next** se inicia el proceso de conexión al servidor y descarga de la extensión seleccionada. Siguiendo con la descarga se inicia la instalación (pulsando sucesivamente **Next**), aceptando el acuerdo de licencia, se selecciona el componente PostGIS para instalar, siguiendo el proceso con **Next**:



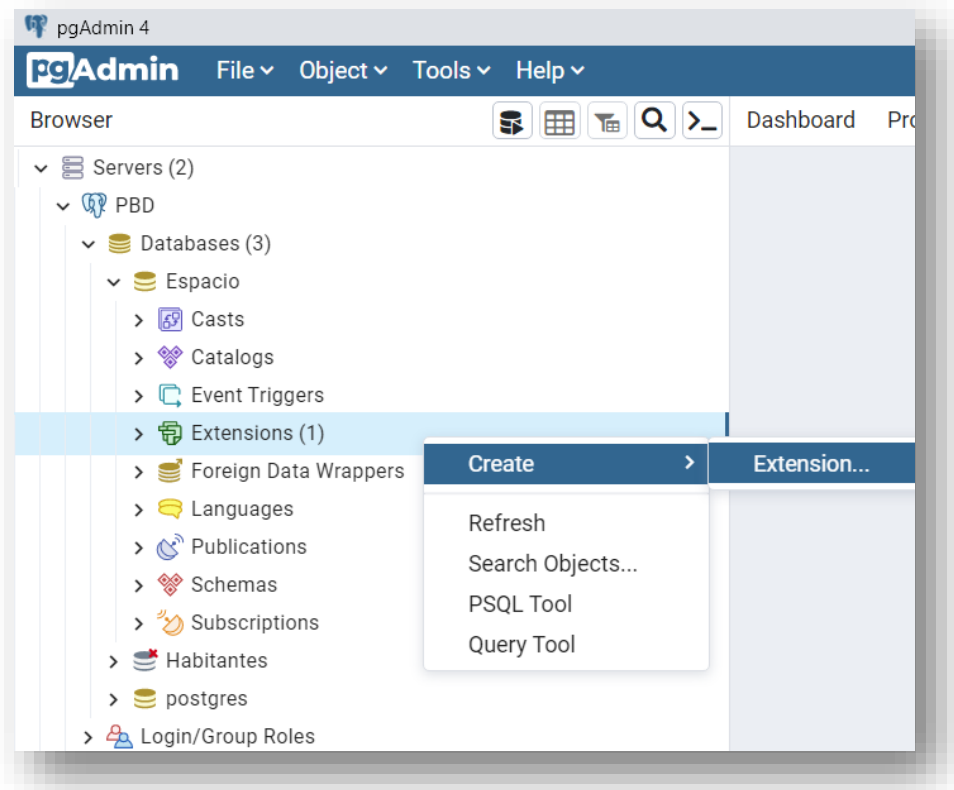
A la cuestión de si se desean registrar variables de entorno como PROJ_LIB, GDAL_DATA... por comodidad, se indicará que “Sí”, hasta finalizar el proceso de instalación:



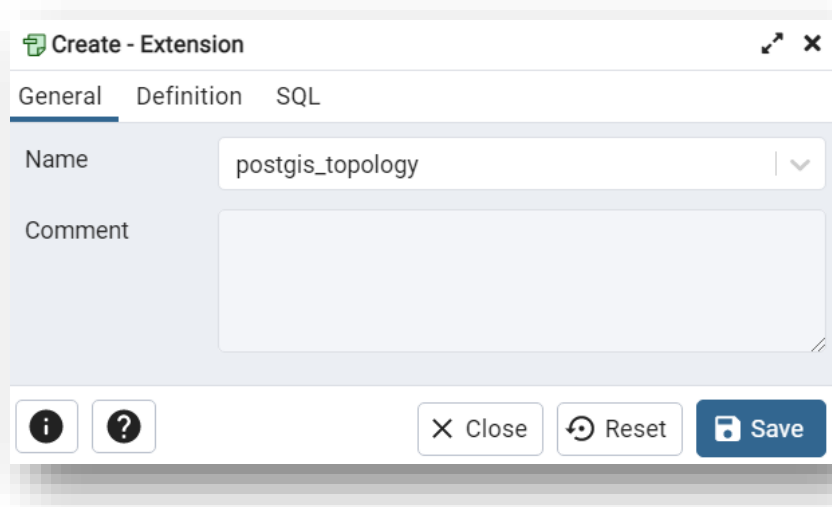
En este momento, ya se está en condiciones de trabajar con datos espaciales sobre **PostgreSQL**. Bastará con arrancar pgAdmin 4 y crear, dentro del servidor **PBD** (configurado en sesiones anteriores) una nueva base de datos. Hasta ahora, sobre el servidor **PBD** se había creado una base de datos **Habitantes**, para trabajar en prácticas anteriores. En este momento, se procederá a crear en el mismo servidor una segunda base de datos, que se llamará **Espacio** (se recuerda que simplemente se hace clic con el botón derecho del ratón sobre el servidor **PBD** para crear una nueva base de datos en él):



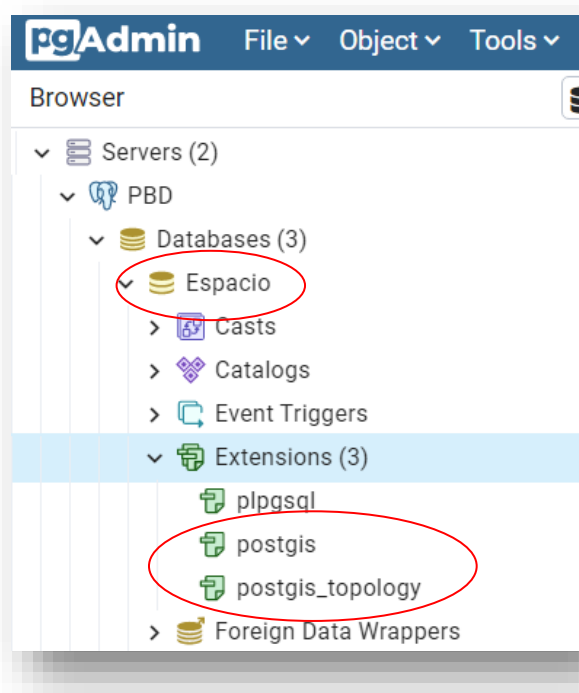
Tras crear la base de datos **Espacio**, en el servidor **PBD**, y tras conectarse a esta base de datos, es preciso añadir la extensión **postGIS** a la misma. Para ello, se hace clic con el botón derecho sobre **Extensions** en la base de datos Espacio, y se selecciona **Create** y luego **Extension**.



En el siguiente formulario, se despliega el menú **Name**, y se seleccionan las extensiones **postGIS** y, tras pulsar Save, se repite el proceso para crear la extensión **postGis_topology**.



De esta forma, se puede apreciar que se han instalado esas dos extensiones espaciales a la base de datos **Espacio** que se acaba de crear:



3. Bases de datos espaciales en PostgreSQL

PostGIS almacena la información geográfica en una columna del tipo GEOMETRY, donde se puede almacenar la geometría en formato WKB (Well-Known Binary).

Las funciones disponibles para tratar con las diferentes geométricas en postGIS se pueden consultar en:

<http://postgis.net/docs/manual-2.0/reference.html>



3.1. Datos espaciales

La información espacial se almacena en tablas espaciales. Así, debe crearse una tabla de tipo espacial, con la sentencia **CREATE TABLE**, como cualquier otra tabla, pero incluyendo un atributo de tipo **GEOMETRY**.

```
CREATE TABLE Mapa (  
  Nombre varchar(32),  
  Geom geometry  
);
```

Otra forma de crear una tabla espacial es añadir una columna espacial a una tabla que ya estuviese creada. Para ello se utiliza la sentencia:

```
AddGeometryColumn (varchar schema_name, varchar table_name,  
varchar column_name, integer srid, varchar type, integer dimension);
```

Donde se identifican los siguientes campos:

- **schema_name**: Nombre del esquema en el que está definida la tabla con la que se va a trabajar. El campo es opcional, si se trabaja sobre el esquema en el que se encuentra la tabla.
- **table_name**: Nombre de la tabla a la que añadir la columna espacial.
- **column_name**: Nombre de la columna espacial que se va a añadir.
- **Srid**: Hace referencia a una entrada en la tabla SPATIAL_REF_SYS.
- **type**: Tipo de geometría.
- **Dimension**: Dimensión en la que vamos a trabajar.

La especificación SFSQL (Simple Features for SQL), estándar para PostGIS, define el modo en el que se representan los objetos del mundo real. SFSQL sólo gestiona representaciones bidimensionales, aunque PostGIS lo ha extendido hasta 3D y 4D.

En los ejemplos que se presentan a continuación, las tablas contienen una mezcla de diferentes geometrías (puntos, líneas, polígonos...). Para trabajar con esta información se pueden usar funciones que leen los metadatos de las geometrías:

- **ST_GeometryType(geometry)** devuelve el tipo de la geometría, que pueden ser:
 - ST_Point
 - ST_Polygon
 - ST_GeometryCollection
 - ST_LineString
- **ST_NDims(geometry)** devuelve el número de dimensiones de la geometría.

Para insertar datos en una tabla espacial se utiliza la sentencia **INSERT**, indicando las distintas geometrías que se deseen insertar.

```
INSERT INTO Mapa VALUES (  
  'Punto', 'POINT (0 0)'  
);
```

```
INSERT INTO Mapa VALUES (  
  'Linea', 'LINESTRING(0 0, 1 1, 2 1, 2 2)'  
);
```



```
INSERT INTO Mapa VALUES (  
  'Poligono',POLYGON((0 0, 1 0, 1 1, 0 1, 0 0))  
);  
  
INSERT INTO Mapa VALUES (  
  'Poligono',POLYGON((0 0, 10 0, 10 10, 0 10, 0 0),(1 1, 1 2, 2 2, 2 1, 1 1))  
);
```

Puntos

Un punto especial representa una localización en la Tierra. El punto se representa mediante coordenadas (que pueden ser 2D, 3D o 4D). Se pueden usar para representar ciudades en un mapa, por ejemplo.

```
SELECT ST_AsText(geom)  
FROM Mapa  
WHERE nombre = 'Punto';  
  
POINT(0 0)
```

Otras funciones espaciales para trabajar con puntos son:

- **ST_X(geometry)** devuelve la coordenada X
- **ST_Y(geometry)** devuelve la coordenada Y

Así, se pueden obtener las coordenadas de un punto de la forma:

```
SELECT ST_X(geom), ST_Y(geom)  
FROM Mapa  
WHERE nombre = 'Punto';
```

LineString

Un linestring es, básicamente, una cadena de puntos. Es adecuada para representar ríos, carreteras... Puede estar cerrada si el punto inicial y el final son el mismo punto.

El código SQL siguiente devolverá la geometría asociada con un linestring (en la columna ST_AsText).

```
SELECT ST_AsText(geom)  
FROM Mapa  
WHERE nombre = 'Linea';  
  
LINESTRING(0 0, 1 1, 2 1, 2 2)
```

Algunas funciones espaciales para trabajar con linestring son:

- **ST_Length(geometry)** devuelve la longitud del linestring
- **ST_StartPoint(geometry)** devuelve la primera coordenada como un punto
- **ST_EndPoint(geometry)** devuelve la última coordenada como un punto
- **ST_NPoints(geometry)** devuelve el número de coordenadas del linestring

Polígonos

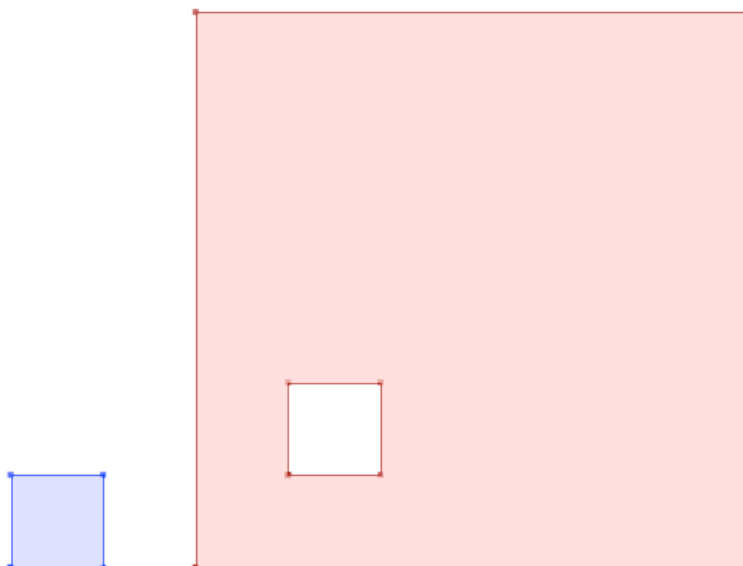
Un polígono es una representación de una superficie. El borde de este polígono se representa como un linestring. Los polígonos se usan para representar objetos con un tamaño importante, como límites de ciudades, parques, edificios...

El siguiente código SQL devolverá la geometría asociada con un linestring (en la columna ST_AsText).

```
SELECT ST_AsText(geom)
FROM Mapa
WHERE nombre = 'Poligono';

POLYGON((0 0, 1 0, 1 1, 0 1, 0 0))
POLYGON((0 0, 10 0, 10 10, 0 10, 0 0),(1 1, 1 2, 2 2, 2 1, 1 1))
```

El primer polígono representa un cuadrado. El segundo, otro cuadrado más grande con un “agujero” en su interior. Obsérvese que hay un doble paréntesis para representar cada polígono.

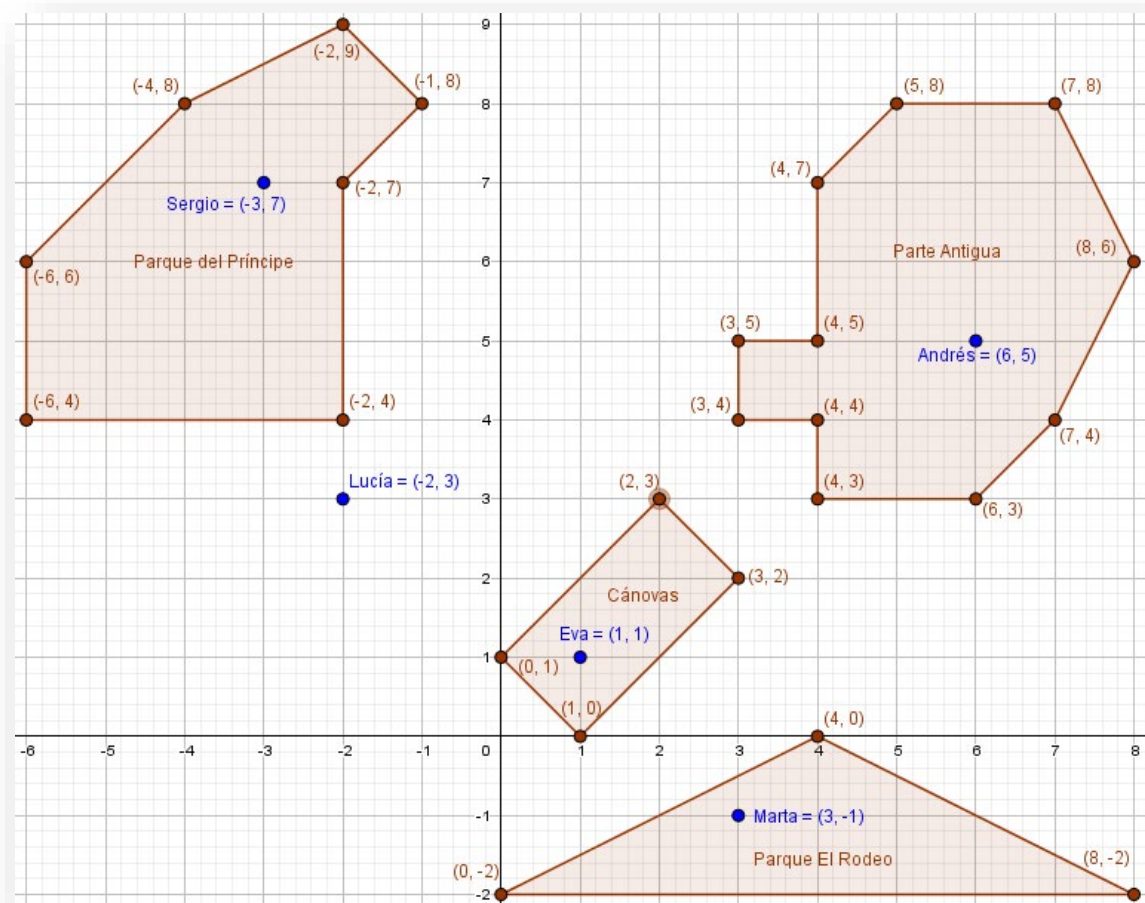


Algunas funciones espaciales para trabajar con polígonos son:

- **ST_Area(geometry)** devuelve el área del polígono
- **ST_NRings(geometry)** devuelve el número de polígonos que contiene una geometría del tipo polígono (generalmente 1, o más si hay “agujeros”)
- **ST_ExteriorRing(geometry)** devuelve el polígono más externo como un linestring
- **ST_InteriorRingN(geometry,n)** devuelve el polígono interno especificado como un linestring
- **ST_Perimeter(geometry)** devuelve la longitud de los polígonos

4. Ejemplo 1: Geometría que contiene puntos y líneas

El primer ejemplo propuesto consiste en una base de datos espacial que incluye **puntos** que corresponden a **personas** y **polígonos** que corresponden a **barrios**. El código correspondiente a este ejemplo se incluye en el fichero “**Espacial_1_postgresql.sql**”.



Se incluyen personas, representadas como puntos (en color azul) y barrios, representados como polígonos (en color marrón).

4.1. Creación e inserción de objetos

Para alojar las localidades y barrios considerados en la figura anterior se crea una tabla espacial. Para el ejemplo actual, se muestra el código para la creación de la tabla y la inserción de los objetos.

```
-- Crear tabla
CREATE TABLE Caceres (
  Nombre VARCHAR(32),
  Tipo VARCHAR(32),
  Geom geometry
);
```

```
-- Insertar personas
INSERT INTO Caceres VALUES(
  'Andrés',
  'Persona',
  'POINT (6 5)'
);
-- ... igual con el resto de personas

-- Insertar barrios
INSERT INTO Caceres VALUES(
  'Parque del Príncipe',
  'Barrio',
  'POLYGON ((-2 9, -4 8, -6 6, -6 4, -2 4, -2 7, -1 8, -2 9))'
);
-- ... igual con el resto de barrios
```

4.2. Consultas

En el ejemplo desarrollado se presentan consultas sobre cuál es la persona o el barrio más cercano a una persona dada, o si una persona se encuentra o no en un barrio.

4.2.1. Consulta del vecino más próximo

La mayor cercanía a una geometría implica realizar una consulta de cercanía localizando la de menor distancia. Se incluyen el ejemplo de encontrar a la persona más cercana a una dada (amigoMasCercano).

```
CREATE OR REPLACE FUNCTION amigoMasCercano(pnombre IN VARCHAR) RETURNS INTEGER AS $$
DECLARE
  geomEntrada Geometry;
  dist NUMERIC;
  distMin NUMERIC;
  tupla Caceres%ROWTYPE;
  nomAmigo Caceres.nombre%TYPE;
  -- Cursor para recuperar las personas
  cursor_persona CURSOR IS
    SELECT *
    FROM Caceres
    WHERE tipo='Persona'; -- Se seleccionan solamente las personas

BEGIN
  RAISE NOTICE '--- amigoMasCercano ---';
  distMin := 9999;

  -- Recuperar la geometria del parametro de entrada
  SELECT Geom INTO geomEntrada
  FROM Caceres
  WHERE nombre = pnombre;

  -- Recorrer todas las personas
  OPEN cursor_persona;
  LOOP
    FETCH cursor_persona INTO tupla;
    IF tupla IS NULL THEN EXIT;
    END IF;
    dist := ST_Distance(tupla.Geom,geomEntrada);

    -- Si estan a menos de la distancia minima...
    IF (dist < distMin) and (tupla.Nombre <> pnombre) THEN
      distMin = dist;
    END IF;
  END LOOP;
END;
```



```
        nomAmigo = tupla.Nombre;
    END IF;
END LOOP;
CLOSE cursor_persona;

RAISE NOTICE 'El amigo más cercano a % es &, a una distancia de %',pnombre,nomAmigo,distMin;
RETURN NULL;
END;
$$ LANGUAGE plpgsql;
```

4.2.2. Función ver()

Para visualizar adecuadamente la información de la tabla espacial, se implementa la función `ver()`, que permite mostrar toda la información de forma adecuada.

```
CREATE OR REPLACE FUNCTION ver() RETURNS INTEGER AS $$
DECLARE
    tupla Caceres%ROWTYPE;

    cursor_ver_Caceres CURSOR IS
        SELECT *
        FROM Caceres;

BEGIN
    RAISE NOTICE '--- VER Caceres ---';
    OPEN cursor_ver_Caceres;
    LOOP
        FETCH cursor_ver_Caceres INTO tupla;
        IF tupla IS NULL THEN EXIT;
        END IF;
        RAISE NOTICE '% - % - % - %',rpad(tupla.nombre,32,' '),rpad(tupla.tipo,32,' '),ST_GeometryType(tupla.Geom),ST_AsText(tupla.Geom);
    END LOOP;
    CLOSE cursor_ver_Caceres;

    RETURN NULL;
END;
$$ LANGUAGE plpgsql;
```

4.3. El módulo principal para el ejemplo 1

El módulo principal para el último ejemplo se presenta a continuación:

```
SELECT Ver();
SELECT amigoMasCercano('Andrés');
SELECT amigoMasCercano('Eva');
SELECT amigoMasCercano('Lucía');

-- Modificar 'Andrés' a coordenada (2,2) dentro Cánovas */
UPDATE Caceres
SET Geom = 'POINT (2 2)'
WHERE nombre='Andrés';

SELECT amigoMasCercano('Andrés');

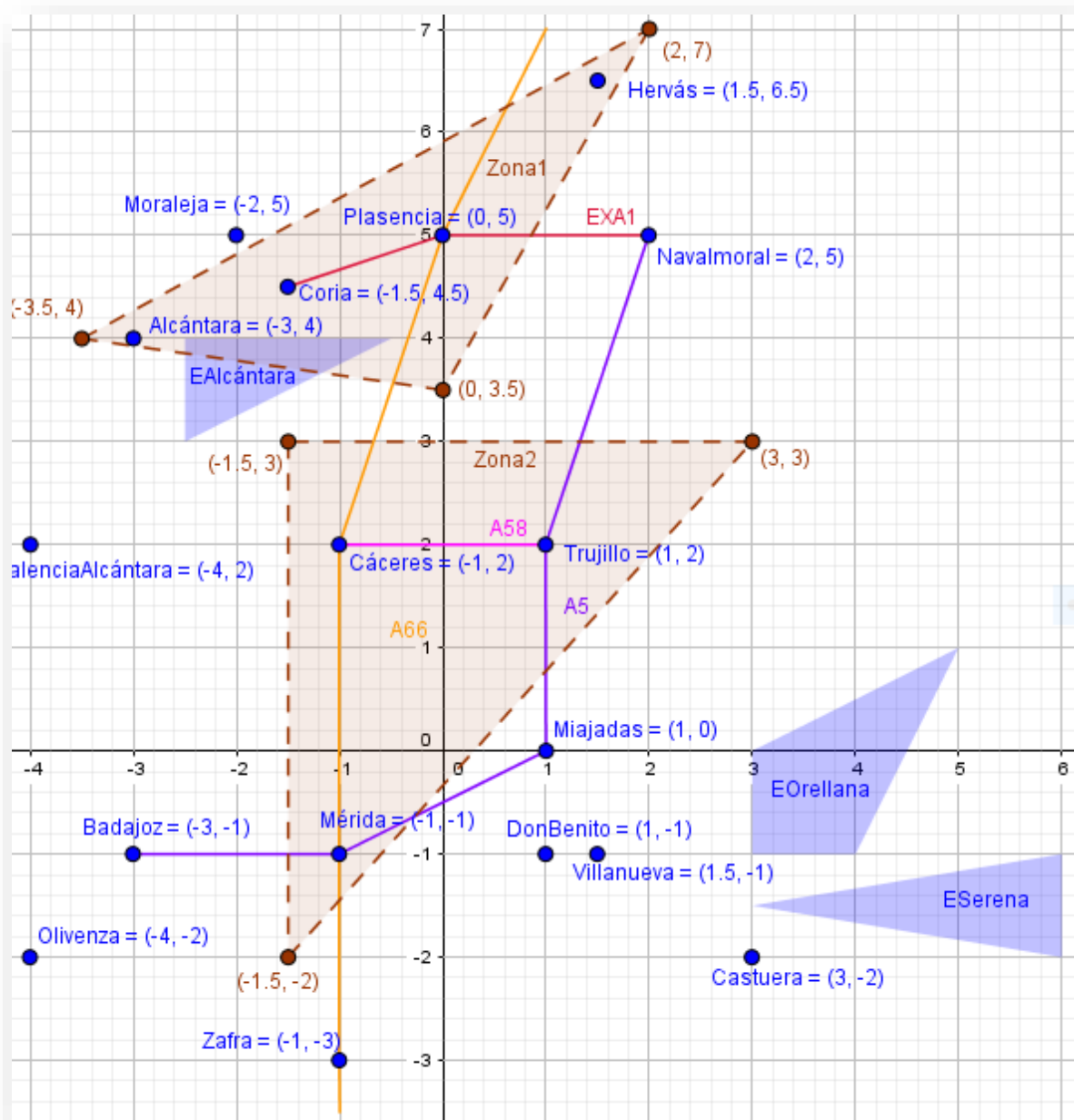
-- Borrado de funciones y tabla */

DROP FUNCTION Ver();
DROP FUNCTION amigoMasCercano();
DROP TABLE Caceres;
```

Tras ejecutar la llamada, el algoritmo devuelve el amigo más cercano a la persona que se pasa como parámetro. Finalmente vendrían las sentencias para borrar todo lo creado.

5. Ejemplo 2: Geometría que contiene puntos, líneas y polígonos

En este ejemplo se representan varias localidades mediante puntos (en color azul) y líneas que representan autovías (en color naranja la A66, en color lila la A5, en color rosa la A58 y en color rojo la EX - A1). También se incluyen polígonos, que representan embalses en color azul (EAlcántara, EOrellana y ESerena) y zonas turísticas en color marrón (Zona1 y Zona2).



6. Ejercicios propuestos

A. Siguiendo el guion de la práctica para el SGBD Oracle, implementar el código equivalente en PostgreSQL a los dos ejemplos anteriores. De este modo, se deben implementar los siguientes dos scripts, que contendrán las funciones que se indican a continuación:

- **Espacial1_postgresql.sql**
 - Ejemplo correspondiente al apartado 4
 - **Tabla:** Cáceres
 - **Geometrías:**
 - Persona – POINT
 - Barrio – POLYGON
 - **Funciones:**
 - amigoMasCercano()
 - barrioMasCercano()
 - enQueBarrio()
- **Espacial2_postgresql.sql**
 - Ejemplo correspondiente al apartado 5
 - **Tabla:** Extremadura
 - **Geometrías:**
 - Localidad – POINT
 - Autovía – LINESTRING
 - Embalse – POLYGON
 - Turismo – POLYGON
 - **Funciones:**
 - locAutovia()
 - radio()
 - embalseMasCercano()
 - embalseMasGrande(): uso de **ST_AREA**

B. Sobre cualquiera de los ejemplos anteriores, se propone añadir, al menos, **tres nuevos procedimientos** en los que se realicen consultas espaciales que no sean del estilo de las propuestas en el presente documento. En los ejemplos se ha hecho uso exclusivamente de las funciones **ST_DISTANCE** y **ST_AREA**. Se propone que se usen nuevas funciones en los procedimientos espaciales desarrollados, consultado la url que se proponía con anterioridad:

<http://postgis.net/docs/manual-2.0/reference.html>

Especialmente, revisar el apartado: 8.9. Spatial Relationships and Measurements

<http://postgis.net/docs/reference.html>

En esta url, revisar apartados 8.11. Spatial Relationships y 8.12. Measurement functions

- C.** Sobre cualquiera de los dos ejemplos anteriores, añadir alguna geometría nueva y realizar alguna operación espacial que la tenga en cuenta.



EXTRA POINT: Con carácter voluntario, se presenta la posibilidad de obtener un *EXTRA POINT* a quienes deseen profundizar más en los temas presentados en este guion práctico. Para ello, en la memoria a entregar, se deberá incluir (además de los apartados obligatorios del guion) un nuevo apartado denominado *EXTRA POINT* e identificado con el logo *BONUS*, donde se presente algún aspecto novedoso o diferenciador con respecto a la propuesta práctica base.

- **Entregables:** debe entregarse
- **OPCIÓN (1) - OBLIGATORIO**
 - (1) El código fuente correspondiente a los ejercicios debidamente resueltos.
Los ejercicios deben implementarse y documentarse **para todos los SGBD**
 - (2) Un PDF en el que se documente:
 - a) Un anexo completo con **las salidas obtenidas tras la ejecución del script completo del código desarrollado. Las salidas obtenidas deben incluirse en la documentación en modo texto y no como una captura de imágenes.**
- **OPCIÓN (2) - OPTATIVO**
 - a) todo el **código fuente** desarrollado y entregado en esta sesión práctica, ampliamente **comentado y documentado**, con las explicaciones necesarias y las **salidas parciales** obtenidas por cada fragmento de código. Si alguna función/procedimiento del código fuente es idéntica en todos los SGBD, es suficiente con explicar esa función/procedimiento una única vez. No es preciso documentar los ejercicios resueltos en este guion y que no formen parte del código del estudiante, de modo que solo es preciso comentar las funciones usadas en los ejercicios propuestos. **El código fuente debe incluirse en la documentación en modo texto y no como una captura de imágenes, tal y como se presenta en los guiones prácticos de la asignatura.**
 - b) Particularmente, deben explicarse detalladamente todos los **aspectos novedosos** que incluyen el tema que se presenta.
 - c) Todas las capturas, salidas, código que se incluyan en la documentación deben ser **perfectamente legibles**, presentándose con la máxima claridad y corrección.